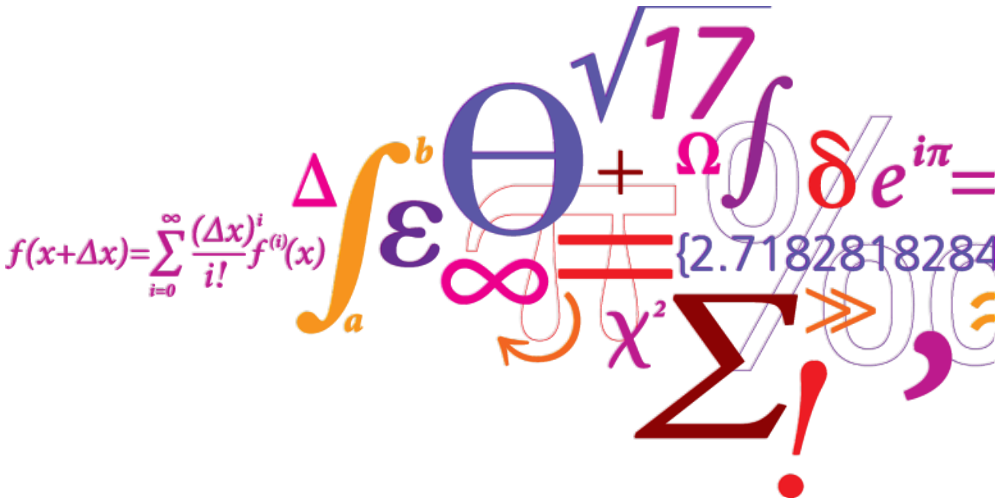
Julia and VS Code

Stefan Røpke



DTU Management
Department of Technology, Management and Economics

What is Julia?

- A programming language like Python, C++, Java, etc.

- Julia is convenient for working with operation research.
  - It is, for example, easy to solve linear programming problems using Julia

- We use it in most of our Operations Research courses

- Free and open source so you can easily use it in a company.

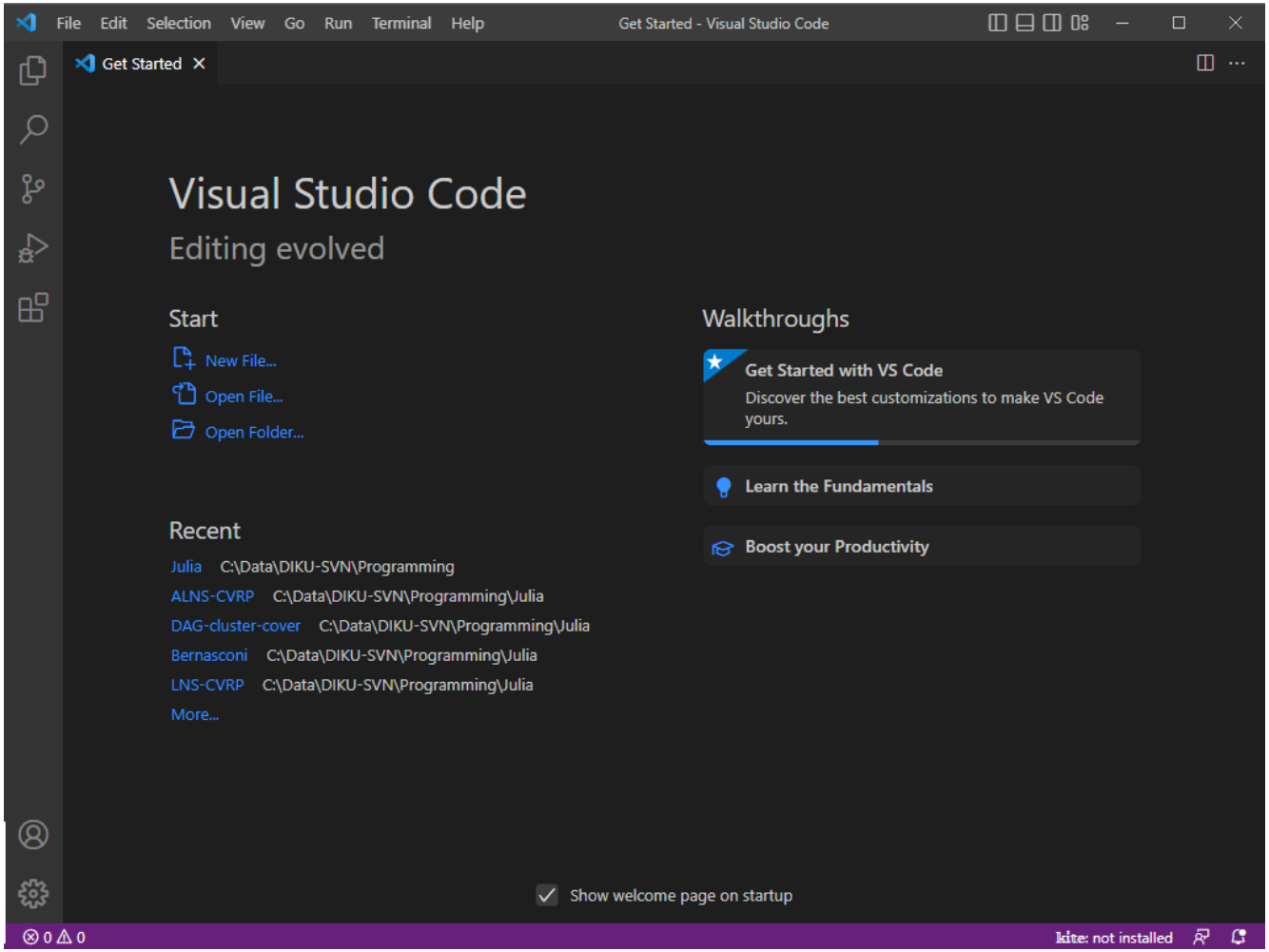
- Why not Python?
  - Julia has a very nice interface to optimization (e.g. LP) solvers.
  - Programs written in Julia are in general faster than those written in Python
  - Julia can be used all the way to your Master Thesis (and your PhD thesis)
  - With Python you would often have to add another programming language along the way.

How to use Julia?

- Install **Julia** and **VS Code** following the instructions on
- https://code.visualstudio.com/docs/languages/julia

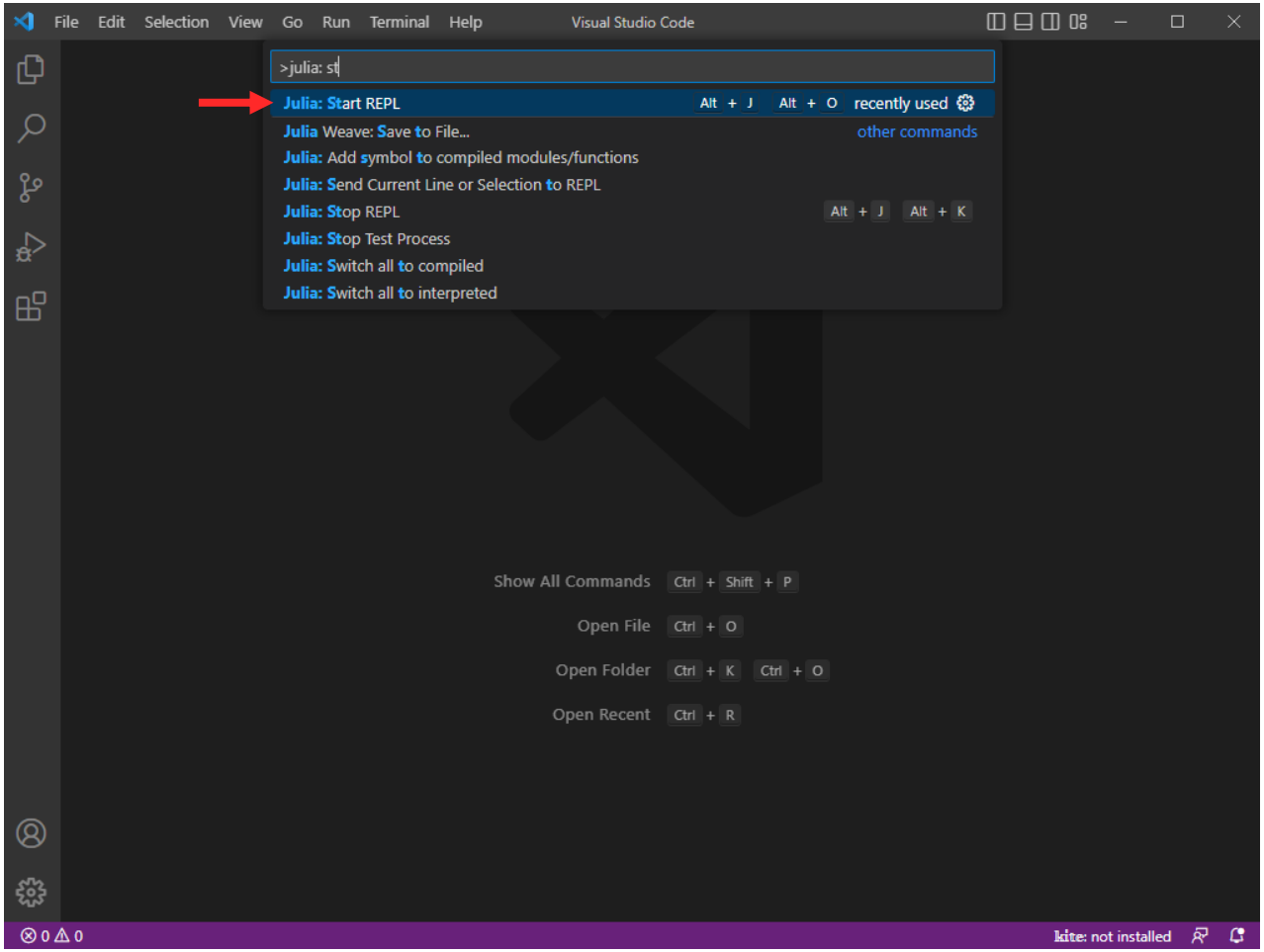
Julia and VS code

- Command palette:
  - Windows: <CTRL>+<SHIFT>+P
  - Mac: <COMMAND>+<SHIFT>+P



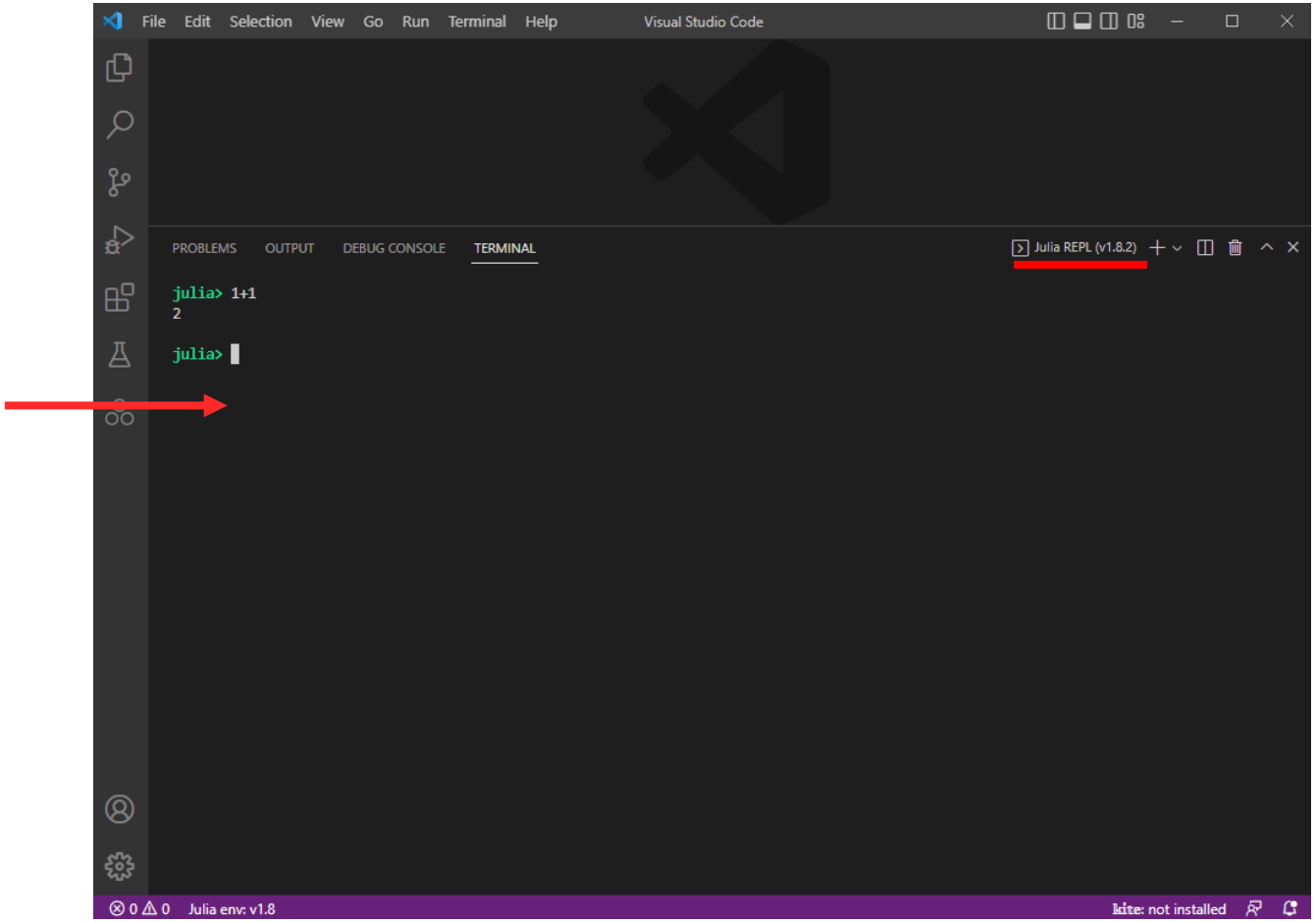
Julia and VS code

- Command palette:
  - Windows: <CTRL>+<SHIFT>+P
  - Mac: <COMMAND>+<SHIFT>+P
- Begin writing "Julia: start REPL". As soon as the right option appears: press enter
- Wait a few seconds



Julia and VS code

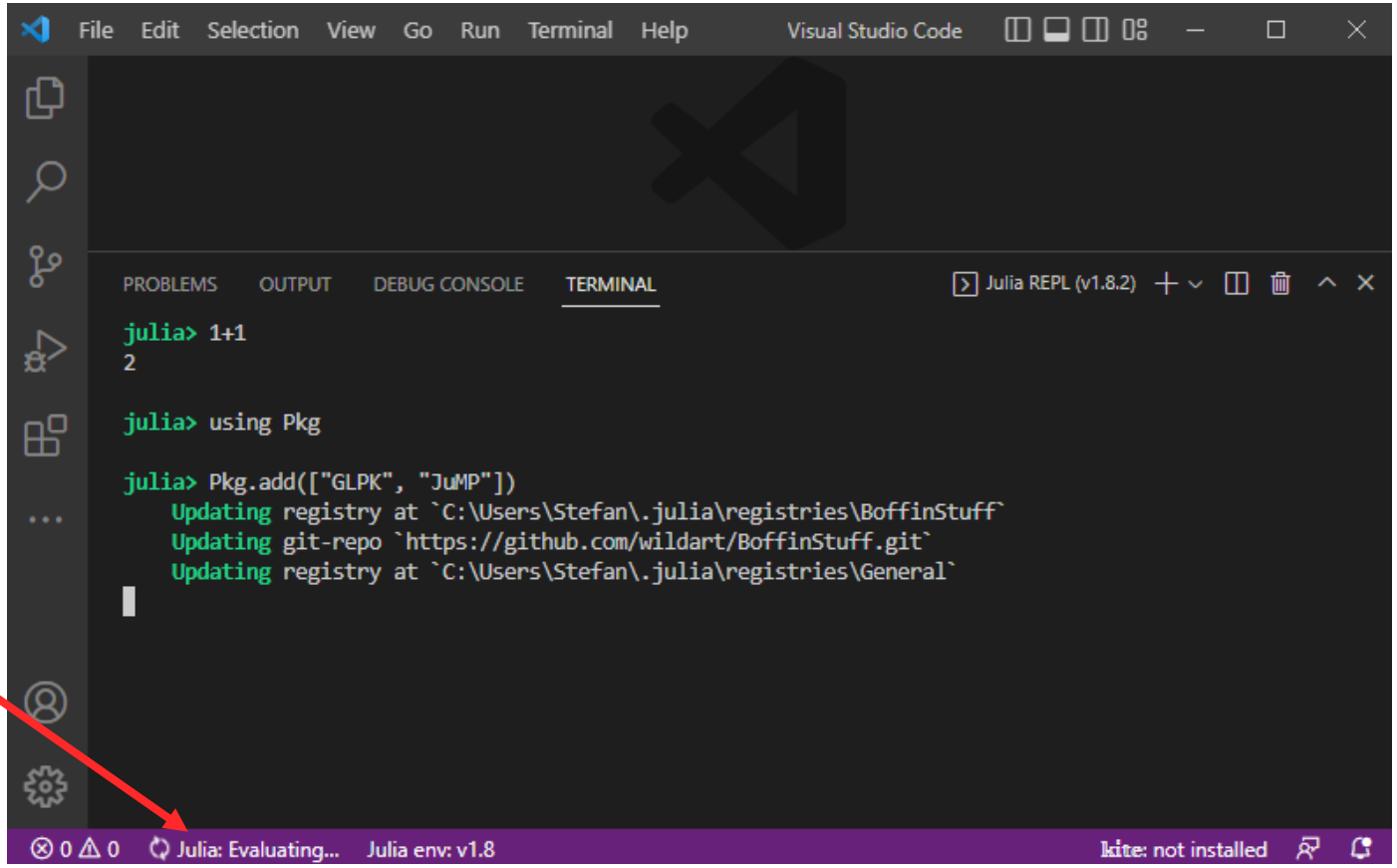
- We have started a Julia REPL where we can write Julia commands
- You can start by writing 1+1 to test that everything is working



Julia and VS code

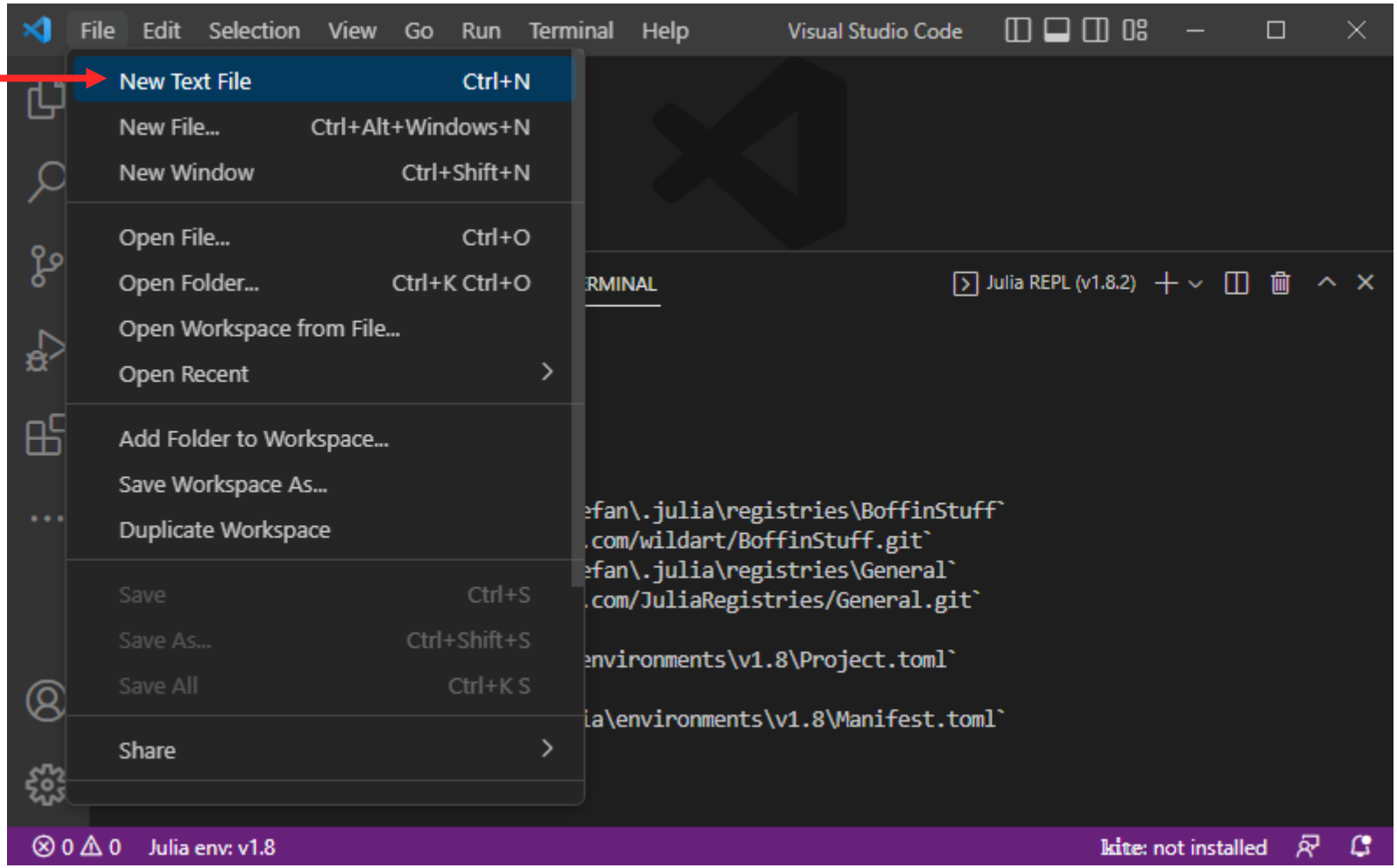
- We need to install two packages that allow us to solve linear programs.
- Type
```
using Pkg
Pkg.add(["GLPK", "JuMP"])
```
- Wait for a few minutes (your output will look different)

You can see that Juila is busy here.



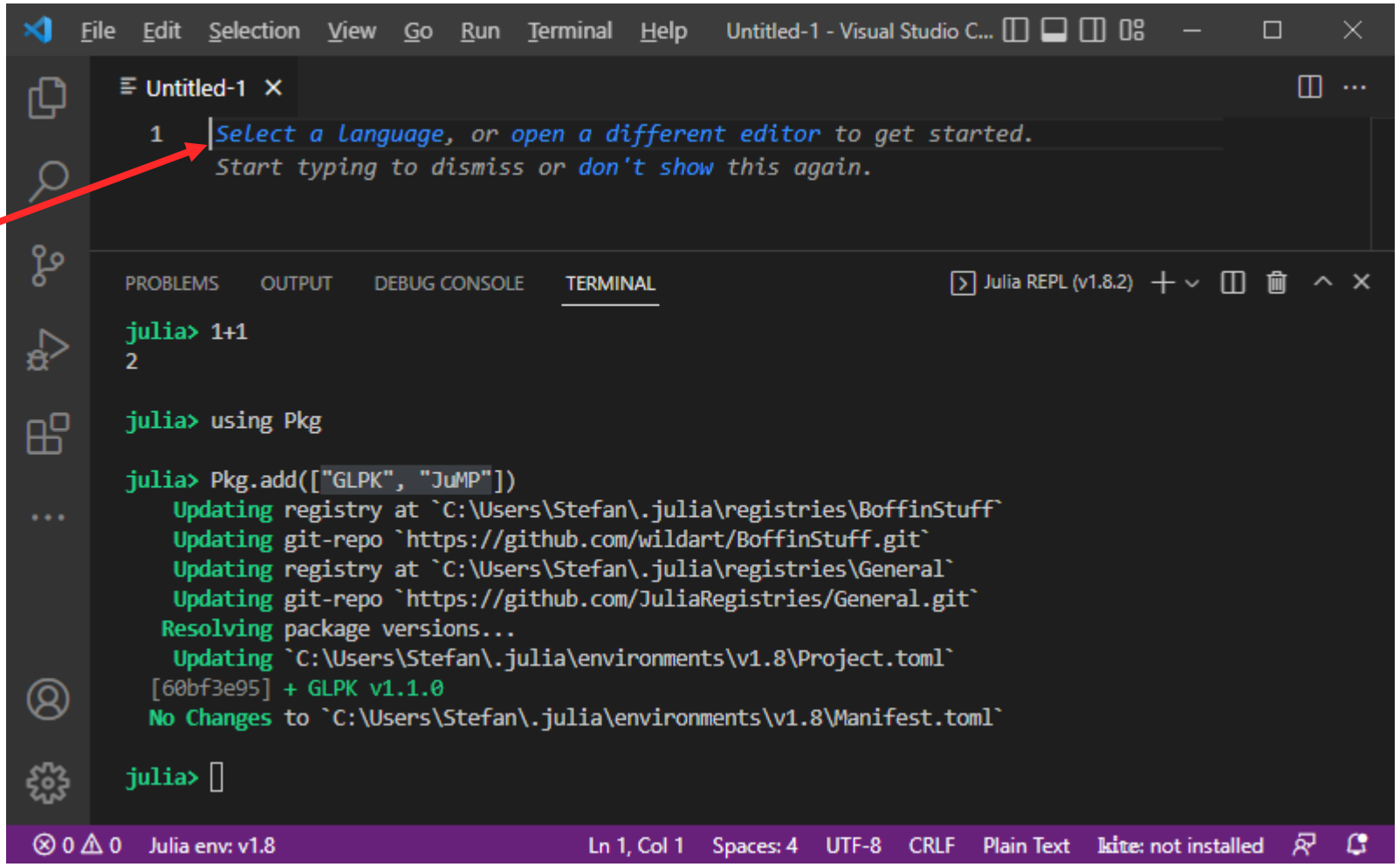
Julia and VS code

- Select
  File -> New Text File
  in the menu



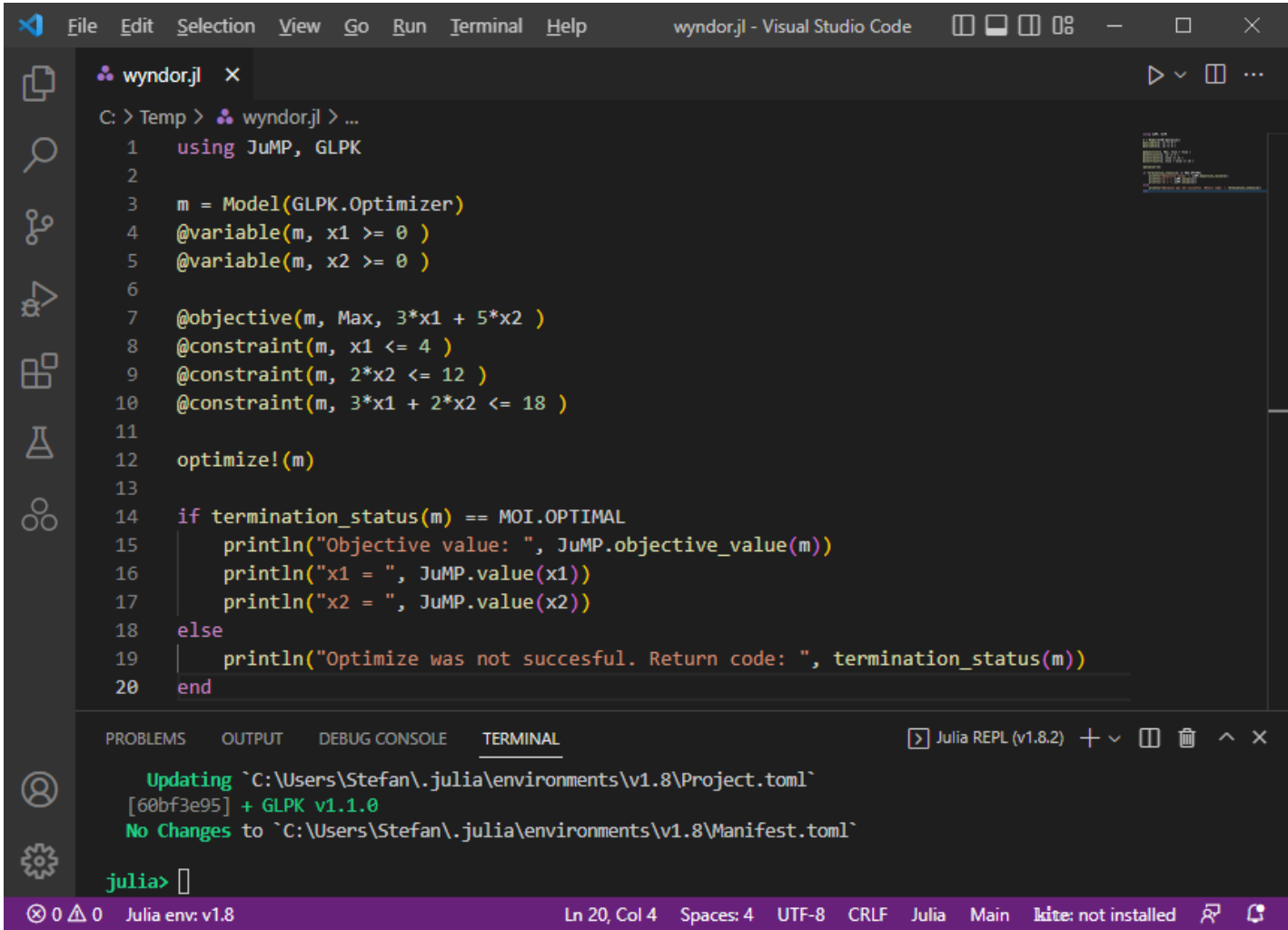
Julia and VS code

- Click “select a language”
- Find and select Julia in the list



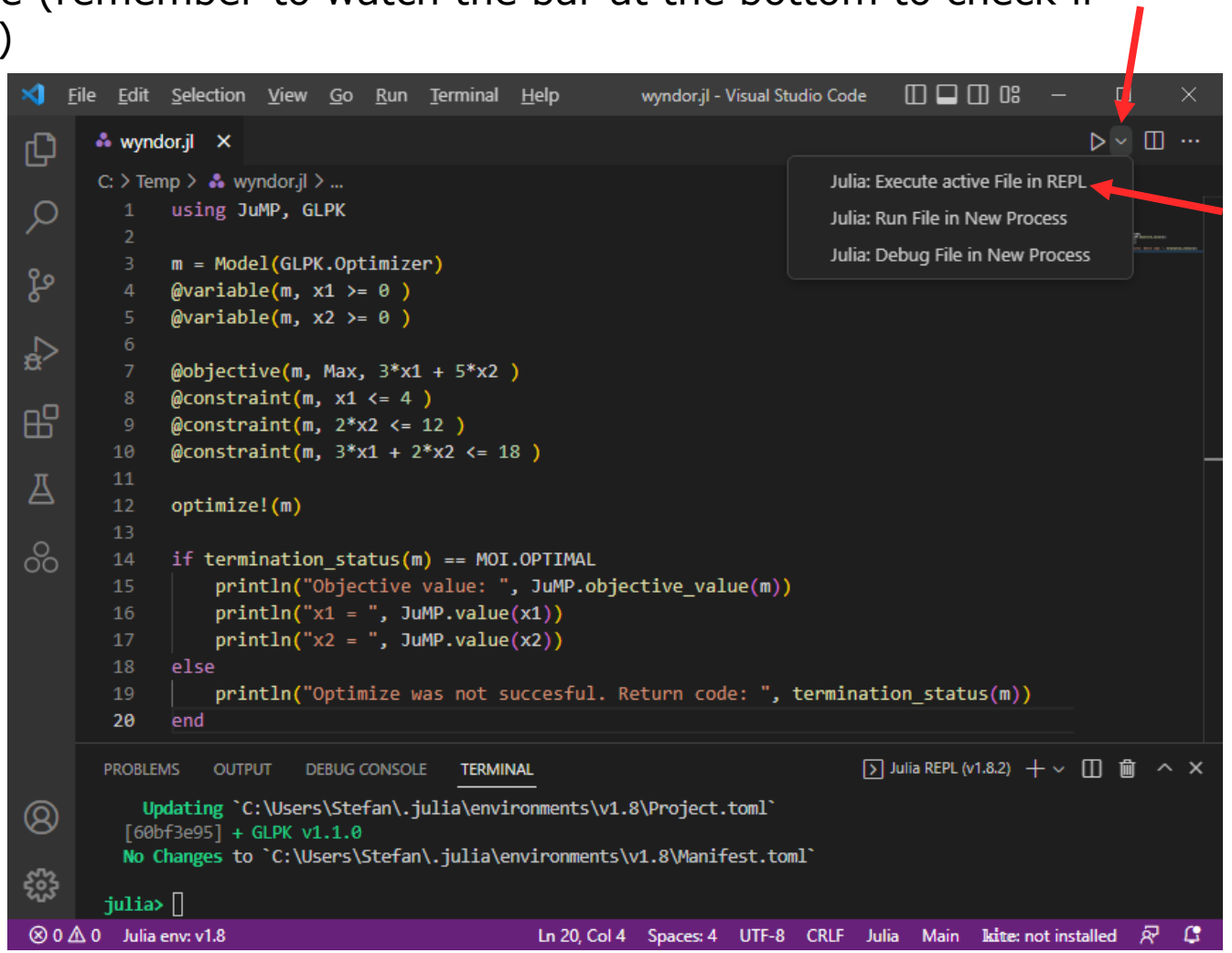
Julia and VS code

- Type in the program below (you can find the code in the file “wyndor.jl” at DTU learn)
- Save to a file (filename should end at “.jl” to indicate it is a Julia file)



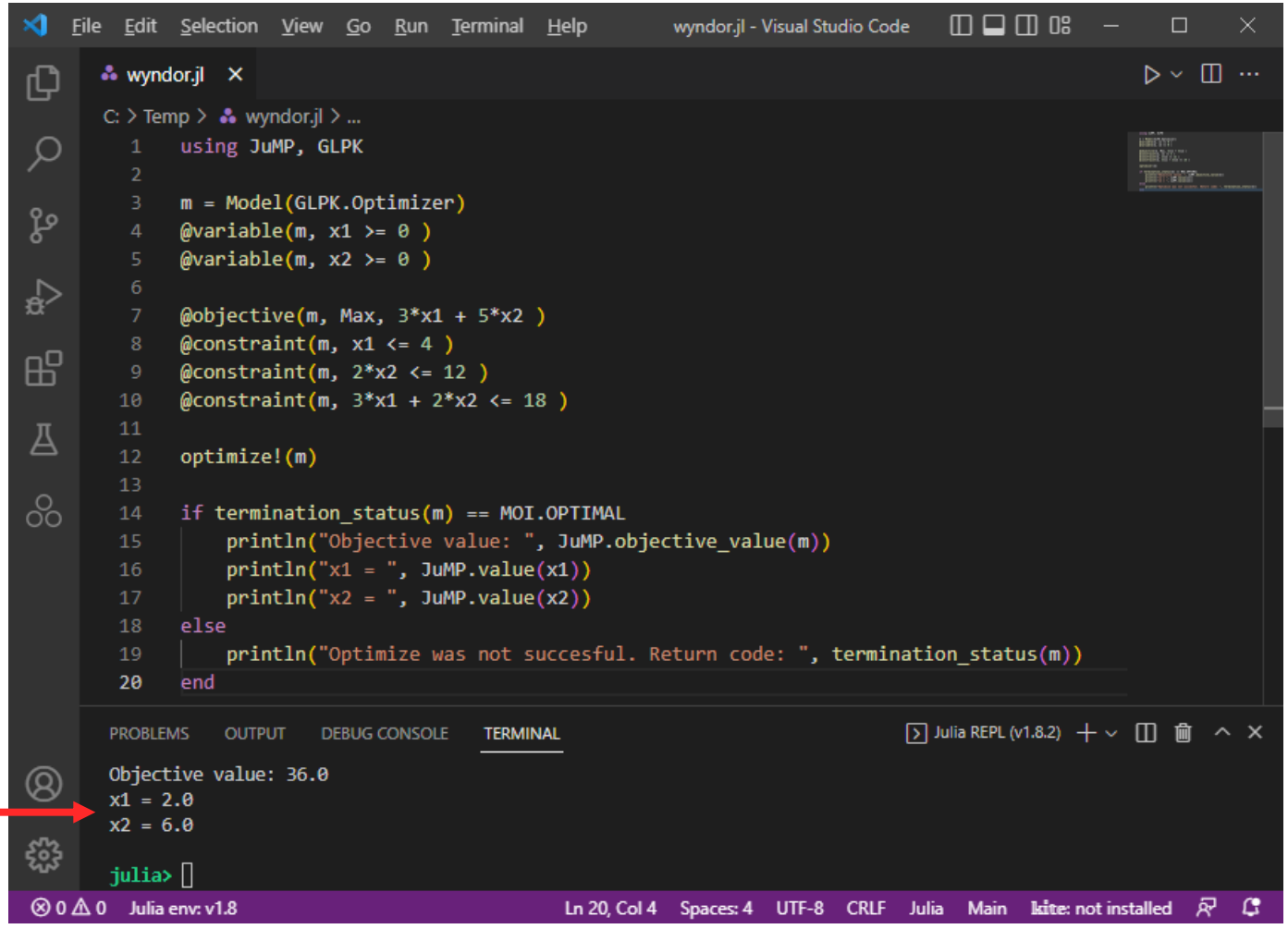
Julia and VS code

- Click the "v" next to the play triangle
- Select "Julia: Execute active File in REPL"
- Wait for a a minute (remember to watch the bar at the bottom to check if Julia is processing)



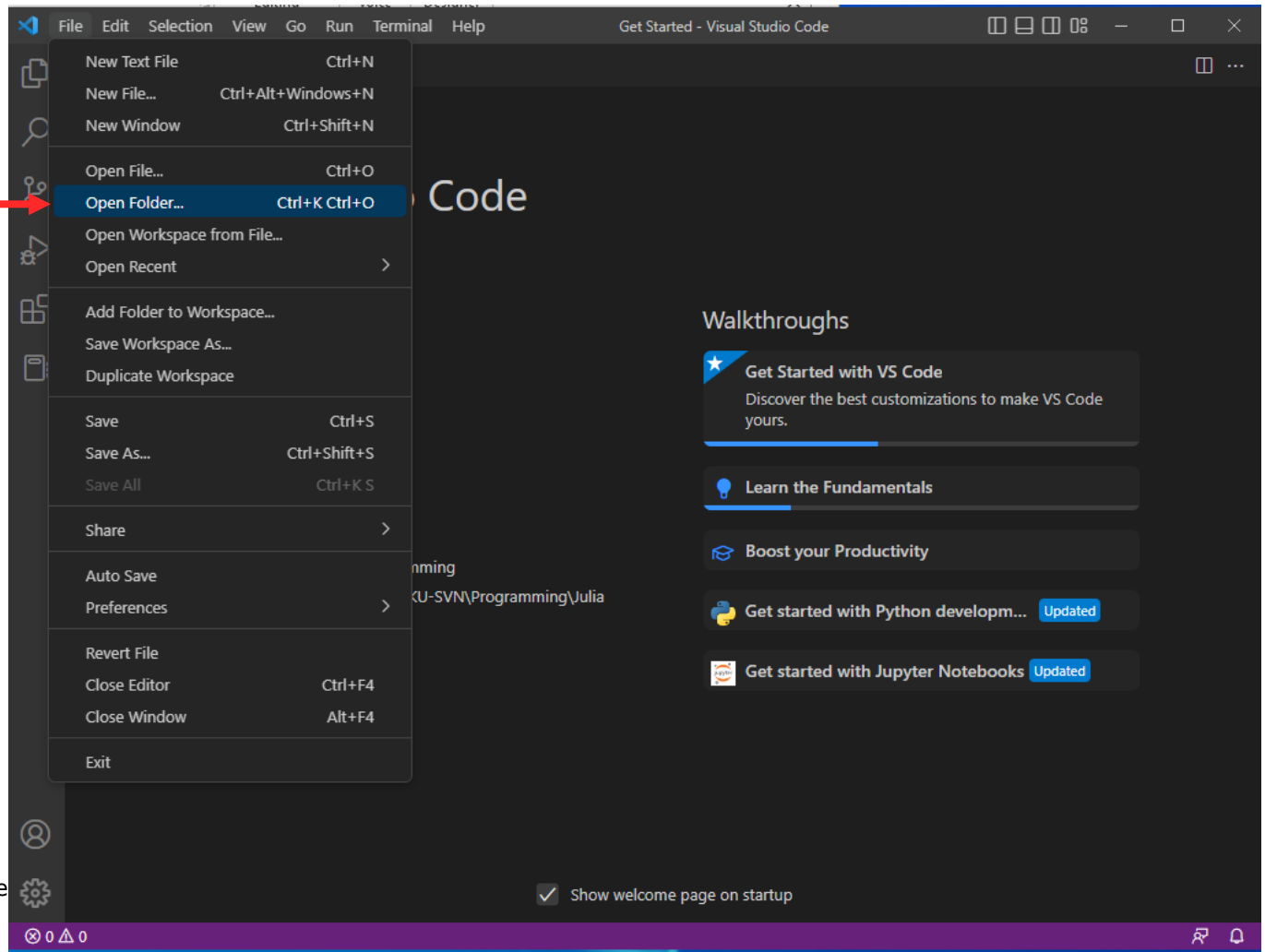
Julia and VS code

- Output from the program appears in the REPL window



If VS code complains about ”missing folder” or similar

- File -> ”open folder”
- Select folder where you plan to store your julia files
- If asked about if your trust the folder. Say yes



Wyndor in Julia

```
using JuMP, GLPK

m = Model(GLPK.Optimizer)
@variable(m, x1 >= 0 )
@variable(m, x2 >= 0 )

@objective(m, Max, 3x1 + 5*x2 )
@constraint(m, x1 <= 4 )
@constraint(m, 2*x2 <= 12 )
@constraint(m, 3*x1 + 2*x2 <= 18 )

optimize!(m)

if termination_status(m) == MOI.OPTIMAL
    println("Objective value: ", JuMP.objective_value(m))
    println("x1 = ", JuMP.value(x1))
    println("x2 = ", JuMP.value(x2))
else
    println("Optimize was not succesful. Return code: ", termination_status(m))
end
```

Tells Julia to include definitions and functions from the two packages we downloaded and creates a new model.

Just copy-paste these two lines when you make a new model

Wyndor in Julia

$$\max Z = 3x_1 + 5x_2$$

subject to

$$\begin{aligned} x_1 &\leq 4 \\ 2x_2 &\leq 12 \\ 3x_1 + 2x_2 &\leq 18 \end{aligned}$$

and

$$x_1 \geq 0,\ x_2 \geq 0$$

```julia
using JuMP, GLPK

m = Model(GLPK.Optimizer)
@variable(m, x1 >= 0 )
@variable(m, x2 >= 0 )

@objective(m, Max, 3x1 + 5*x2 )
@constraint(m, x1 <= 4 )
@constraint(m, 2*x2 <= 12 )
@constraint(m, 3*x1 + 2*x2 <= 18 )

optimize!(m)

if termination_status(m) == MOI.OPTIMAL
    println("Objective value: ", JuMP.objective_value(m))
    println("x1 = ", JuMP.value(x1))
    println("x2 = ", JuMP.value(x2))
else
    println("Optimize was not succesful. Return code: ", termination_status(m))
end
```

Here we define the variables for the model

Wyndor in Julia

$$\max Z = 3x_1 + 5x_2$$

subject to

$$\begin{aligned} x_1 &\leq 4 \\ 2x_2 &\leq 12 \\ 3x_1 + 2x_2 &\leq 18 \end{aligned}$$

and

$$x_1 \geq 0,\ x_2 \geq 0$$

```julia
using JuMP, GLPK

m = Model(GLPK.Optimizer)
@variable(m, x1 >= 0 )
@variable(m, x2 >= 0 )

@objective(m, Max, 3x1 + 5*x2 )
@constraint(m, x1 <= 4 )
@constraint(m, 2*x2 <= 12 )
@constraint(m, 3*x1 + 2*x2 <= 18 )

optimize!(m)

if termination_status(m) == MOI.OPTIMAL
    println("Objective value: ", JuMP.objective_value(m))
    println("x1 = ", JuMP.value(x1))
    println("x2 = ", JuMP.value(x2))
else
    println("Optimize was not succesful. Return code: ", termination_status(m))
end
```

Here we define the objective function.

Wyndor in Julia

$$\max Z = 3x_1 + 5x_2$$

subject to

$$\begin{aligned} x_1 &\leq 4 \\ 2x_2 &\leq 12 \\ 3x_1 + 2x_2 &\leq 18 \end{aligned}$$

and

$$x_1 \geq 0,\ x_2 \geq 0$$

```julia
using JuMP, GLPK

m = Model(GLPK.Optimizer)
@variable(m, x1 >= 0 )
@variable(m, x2 >= 0 )

@objective(m, Max, 3x1 + 5*x2 )
@constraint(m, x1 <= 4 )
@constraint(m, 2*x2 <= 12 )
@constraint(m, 3*x1 + 2*x2 <= 18 )

optimize!(m)

if termination_status(m) == MOI.OPTIMAL
    println("Objective value: ", JuMP.objective_value(m))
    println("x1 = ", JuMP.value(x1))
    println("x2 = ", JuMP.value(x2))
else
    println("Optimize was not succesful. Return code: ", termination_status(m))
end
```

Here we define constraints
Equality constraints are written ==
Greater-than-equal are written >=

Wyndor in Julia

$$\max Z = 3x_1 + 5x_2$$

subject to

$$\begin{aligned} x_1 &\leq 4 \\ 2x_2 &\leq 12 \\ 3x_1 + 2x_2 &\leq 18 \end{aligned}$$

and

$$x_1 \geq 0,\ x_2 \geq 0$$

```julia
using JuMP, GLPK

m = Model(GLPK.Optimizer)
@variable(m, x1 >= 0 )
@variable(m, x2 >= 0 )

@objective(m, Max, 3x1 + 5*x2 )
@constraint(m, x1 <= 4 )
@constraint(m, 2*x2 <= 12 )
@constraint(m, 3*x1 + 2*x2 <= 18 )

optimize!(m)

if termination_status(m) == MOI.OPTIMAL
    println("Objective value: ", JuMP.objective_value(m))
    println("x1 = ", JuMP.value(x1))
    println("x2 = ", JuMP.value(x2))
else
    println("Optimize was not succesful. Return code: ", termination_status(m))
end
```

We ask Julia / GLPK to solve the model

Wyndor in Julia

$$\max Z = 3x_1 + 5x_2$$

subject to

$$\begin{aligned} x_1 &\leq 4 \\ 2x_2 &\leq 12 \\ 3x_1 + 2x_2 &\leq 18 \end{aligned}$$

and

$$x_1 \geq 0,\ x_2 \geq 0$$

```julia
using JuMP, GLPK

m = Model(GLPK.Optimizer)
@variable(m, x1 >= 0 )
@variable(m, x2 >= 0 )

@objective(m, Max, 3x1 + 5*x2 )
@constraint(m, x1 <= 4 )
@constraint(m, 2*x2 <= 12 )
@constraint(m, 3*x1 + 2*x2 <= 18 )

optimize!(m)

if termination_status(m) == MOI.OPTIMAL
    println("Objective value: ", JuMP.objective_value(m))
    println("x1 = ", JuMP.value(x1))
    println("x2 = ", JuMP.value(x2))
else
    println("Optimize was not succesful. Return code: ", termination_status(m))
end
```

Check if model was solved ok.

Wyndor in Julia

$$\max Z = 3x_1 + 5x_2$$

subject to

$$\begin{aligned} x_1 &\leq 4 \\ 2x_2 &\leq 12 \\ 3x_1 + 2x_2 &\leq 18 \end{aligned}$$

and

$$x_1 \geq 0,\ x_2 \geq 0$$

```julia
using JuMP, GLPK

m = Model(GLPK.Optimizer)
@variable(m, x1 >= 0 )
@variable(m, x2 >= 0 )

@objective(m, Max, 3x1 + 5*x2 )
@constraint(m, x1 <= 4 )
@constraint(m, 2*x2 <= 12 )
@constraint(m, 3*x1 + 2*x2 <= 18 )

optimize!(m)

if termination_status(m) == MOI.OPTIMAL
    println("Objective value: ", JuMP.objective_value(m))
    println("x1 = ", JuMP.value(x1))
    println("x2 = ", JuMP.value(x2))
else
    println("Optimize was not succesful. Return code: ", termination_status(m))
end
```

We print the optimal objective value and optimal solution.

Knapsack example

Parameters and sets:
n : number of items.
$N = \{1, \ldots, n\}$: set of all items.
p_i : profit of selecting item i ($i \in N$).
w_i : weight of item i ($i \in N$).
c : knapsack capacity.
Decision variables:
x_i : 1 if item i is added to the knapsack, 0 otherwise.
The model:

$$\max \sum_{i \in N} p_i x_i$$

subject to

$$\sum_{i \in N} w_i x_i \leq c$$

$$x_i \in \{0, 1\} \quad \forall i \in N$$

```
using JuMP, GLPK

n=10
p=[10, 5, 4, 6, 11, 1, 9, 22, 15, 3]
w=[158, 118, 205, 71, 158, 142, 95, 252, 158, 122]
c=866

m = Model(GLPK.Optimizer)
@variable(m, x[1:n], Bin )

@objective(m, Max, sum(p[j] * x[j] for j=1:n) )
@constraint(m, sum(w[j] * x[j] for j=1:n) <= c)

optimize!(m)

println("Objective value: ", JuMP.objective_value(m))
println("x = ", JuMP.value.(x))
```

Define Data

Knapsack example

Parameters and sets:
n : number of items.
$N = \{1, \ldots, n\}$: set of all items.
p_i : profit of selecting item i ($i \in N$).
w_i : weight of item i ($i \in N$).
c : knapsack capacity.
Decision variables:
x_i : 1 if item i is added to the knapsack, 0 otherwise.
The model:

$$\max \sum_{i \in N} p_i x_i$$

subject to

$$\sum_{i \in N} w_i x_i \leq c$$

$$x_i \in \{0, 1\} \quad \forall i \in N$$

```
using JuMP, GLPK

n=10
p=[10, 5, 4, 6, 11, 1, 9, 22, 15, 3]
w=[158, 118, 205, 71, 158, 142, 95, 252, 158, 122]
c=866

m = Model(GLPK.Optimizer)
@variable(m, x[1:n], Bin )

@objective(m, Max, sum(p[j] * x[j] for j=1:n) )
@constraint(m, sum(w[j] * x[j] for j=1:n) <= c)

optimize!(m)

println("Objective value: ", JuMP.objective_value(m))
println("x = ", JuMP.value.(x))
```

Define n binary variables

Knapsack example

Parameters and sets:
n : number of items.
$N = \{1, \ldots, n\}$: set of all items.
p_i : profit of selecting item i ($i \in N$).
w_i : weight of item i ($i \in N$).
c : knapsack capacity.
Decision variables:
x_i : 1 if item i is added to the knapsack, 0 otherwise.
The model:

$$\max \sum_{i \in N} p_i x_i$$

subject to

$$\sum_{i \in N} w_i x_i \leq c$$

$$x_i \in \{0, 1\} \quad \forall i \in N$$

```
using JuMP, GLPK

n=10
p=[10, 5, 4, 6, 11, 1, 9, 22, 15, 3]
w=[158, 118, 205, 71, 158, 142, 95, 252, 158, 122]
c=866

m = Model(GLPK.Optimizer)
@variable(m, x[1:n], Bin )

@objective(m, Max, sum(p[j] * x[j] for j=1:n) )
@constraint(m, sum(w[j] * x[j] for j=1:n) <= c)

optimize!(m)

println("Objective value: ", JuMP.objective_value(m))
println("x = ", JuMP.value.(x))
```

Define objective function and constraints.

Notice how constraints are written in a generic way, independent of data

Capacitated Facility location

$$\min \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} + \sum_{i \in I} f_i y_i$$

subject to

$$\sum_{i \in I} x_{ij} = 1 \qquad \forall j \in J$$

$$\sum_{j \in J} d_j x_{ij} \leq Q y_i \qquad \forall i \in I$$

$$x_{ij} \in \{0,1\} \qquad \forall i \in I, j \in J$$

$$y_i \in \{0,1\} \qquad \forall i \in I$$

Parameters	
f_i	cost of opening facility $i \in I$
c_{ij}	cost of serving customer $j \in J$ using facility $i \in I$
Q_i	capacity of facility $i \in I$.
d_j	demand of customer $j \in J$
Decision variables	
$y_i \in \{0,1\}$	1 if facility $i \in I$ is opened
$x_{ij} \in \{0,1\}$	1 if customer j is served by facility i. 0 otherwise.

```
using JuMP, GLPK
m=3     # m:  number of facilities
n=5     # n: number of customers
f=[1000 1000 1000]
d=[12 10 11 15 15]
Q=40
c = [   2  4  1  8  3;
        10  7  5  1  7;
        8  2  3  1  1]
```

Define Data,
notice how we construct a matrix

```
IP = Model(GLPK.Optimizer)
@variable(IP, x[1:m, 1:n], Bin )
@variable(IP, y[1:m], Bin )
@objective(IP, Min, sum(c[i,j] * x[i,j] for i=1:m for j=1:n) + sum(f[i]*y[i] for i=1:m))
@constraint(IP, [j=1:n], sum(x[i,j] for i=1:m) == 1)
@constraint(IP, [i=1:m], sum(d[j]*x[i,j] for j=1:n) <= Q*y[i])
optimize!(IP)

println("Objective value: ", JuMP.objective_value(IP))
println("x = ", value.(x))
println("y =  ", value.(y))
```

Capacitated Facility location

$$\min \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} + \sum_{i \in I} f_i y_i$$

subject to

$$\sum_{i \in I} x_{ij} = 1 \qquad \forall j \in J$$

$$\sum_{j \in J} d_j x_{ij} \leq Q y_i \qquad \forall i \in I$$

$$x_{ij} \in \{0,1\} \quad \forall i \in I, j \in J$$

$$y_i \in \{0,1\} \quad \forall i \in I$$

Parameters	
f_i	cost of opening facility $i \in I$
c_{ij}	cost of serving customer $j \in J$ using facility $i \in I$
Q_i	capacity of facility $i \in I$.
d_j	demand of customer $j \in J$
Decision variables	
$y_i \in \{0,1\}$	1 if facility $i \in I$ is opened
$x_{ij} \in \{0,1\}$	1 if customer j is served by facility i. 0 otherwise.

```
using JuMP, GLPK
m=3     # m:  number of facilities
n=5     # n: number of customers
f=[1000 1000 1000]
d=[12 10 11 15 15]
Q=40
c = [   2  4  1  8  3;
        10  7  5  1  7;
        8  2  3  1  1]

IP = Model(GLPK.Optimizer)
@variable(IP, x[1:m, 1:n], Bin )
@variable(IP, y[1:m], Bin )
@objective(IP, Min, sum(c[i,j] * x[i,j] for i=1:m for j=1:n) + sum(f[i]*y[i] for i=1:m))
@constraint(IP, [j=1:n], sum(x[i,j] for i=1:m) == 1)
@constraint(IP, [i=1:m], sum(d[j]*x[i,j] for j=1:n) <= Q*y[i])
optimize!(IP)

println("Objective value: ", JuMP.objective_value(IP))
println("x = ", value.(x))
println("y =  ", value.(y))
```

Define variables
Notice that x has two indices

Capacitated Facility location

$$\min \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij} + \sum_{i \in I} f_i y_i$$

subject to

$$\sum_{i \in I} x_{ij} = 1 \qquad \forall j \in J$$

$$\sum_{j \in J} d_j x_{ij} \leq Q y_i \qquad \forall i \in I$$

$$x_{ij} \in \{0,1\} \quad \forall i \in I, j \in J$$

$$y_i \in \{0,1\} \quad \forall i \in I$$

Parameters	
f_i	cost of opening facility $i \in I$
c_{ij}	cost of serving customer $j \in J$ using facility $i \in I$
Q_i	capacity of facility $i \in I$.
d_j	demand of customer $j \in J$
Decision variables	
$y_i \in \{0,1\}$	1 if facility $i \in I$ is opened
$x_{ij} \in \{0,1\}$	1 if customer j is served by facility i. 0 otherwise.

```
using JuMP, GLPK
m=3     # m:  number of facilities
n=5     # n: number of customers
f=[1000 1000 1000]
d=[12 10 11 15 15]
Q=40
c = [   2  4  1  8  3;
        10  7  5  1  7;
        8  2  3  1  1]

IP = Model(GLPK.Optimizer)
@variable(IP, x[1:m, 1:n], Bin )
@variable(IP, y[1:m], Bin )
@objective(IP, Min, sum(c[i,j] * x[i,j] for i=1:m for j=1:n) + sum(f[i]*y[i] for i=1:m))
@constraint(IP, [j=1:n], sum(x[i,j] for i=1:m) == 1)
@constraint(IP, [i=1:m], sum(d[j]*x[i,j] for j=1:n) <= Q*y[i])
optimize!(IP)

println("Objective value: ", JuMP.objective_value(IP))
println("x = ", value.(x))
println("y =  ", value.(y))
```

Define constraints
The first statement construct a constraint for all j = 1 to n

The second statement construct a constraint for all i = 1 to m

That’s it ☺ Let’s use it in practice